CSCI 235 Midterm Reference Sheet

Tue Oct 27, 2026 • 8:30–9:45 • Hunter College

Covers P1 (MiniVector), P2 (Maze Solver), and everything through DFS/BFS. Not covered: state-space search, permutations, Valgrind.

1. Classes and Polymorphism

virtual. With `virtual`, the *actual object* decides which function runs (at run time). Without it, the *pointer/reference type* decides (at compile time).

```
struct Shape { virtual string name() const;
               virtual ~Shape(); };
struct Circle : Shape {
  string name() const override; };
Shape* p = new Circle;
p->name();   // Circle's (virtual)
```

- A **pure virtual function**, `virtual double area() const = 0;`, makes the class **abstract**: cannot create objects of it, only pointers/references. A derived class must override it to be concrete.
- If an object may be deleted through a base pointer, the base class needs a **virtual destructor**. Otherwise deleting through that pointer is **undefined behavior** (the derived destructor may not run).
- **Override**: same signature, new behavior in the derived class. **Overload**: same name, *different parameters*.
- **Slicing**: `Shape s = c;` copies only the `Shape` part of `c`; the Circle part is lost, so no polymorphism. Use pointers/references.

Copy constructor	Copy assignment
Builds a *new* object.	Overwrites an *existing* object.
`T b(a);` `T b = a;`	`b = a;`
Nothing old to clean up.	Must avoid leaking the old memory; check self-assignment; **return `*this`**.

```
T& operator=(const T& o) {
  if (this != &o) {
    T* nd = new T[o.cap_];      // 1. allocate
    for (int i = 0; i < o.size_; i++)
      nd[i] = o.data_[i];       // 2. deep copy
    delete[] data_;             // 3. release old
    data_ = nd;
    size_ = o.size_; cap_ = o.cap_;
  }
  return *this;                 // 4. return *this
}
```

2. Templates, MiniVector, Memory

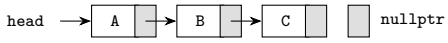
- `template <typename T>` lets one class/function work for any type; the compiler generates a version per type used.
- **Rule of Three**: if a class owns heap memory, define the **destructor**, **copy constructor**, and **copy assignment**.
- **Deep copy**: allocate new memory and copy the elements. A **shallow copy** (copying only the pointer) makes two objects share one array, so both destructors `delete` it: **double delete**.
- `new` pairs with `delete`; `new[]` pairs with `delete[]`. Each `new` must eventually be released once (no leak, no double delete).

Growth by doubling (when `size == capacity`):

```
int newCap = (cap_ == 0) ? 1 : 2 * cap_;
T* bigger = new T[newCap];
for (int i = 0; i < size_; i++)
  bigger[i] = data_[i];
delete[] data_;
data_ = bigger;  cap_ = newCap;
data_[size_++] = x;
```

- One resize costs $O(n)$ (copy n items), but resizes are rare: the total copies for n pushes is $< 2n$, so `push_back` is **amortized** $O(1)$, whatever the initial capacity.
- Resizing by a fixed amount (e.g. $+10$) is **not** amortized $O(1)$; it gives $O(n)$ per push on average.
- Indexing `v[i]`: $O(1)$. Insert/erase in the middle: $O(n)$ (shift).

3. Singly Linked List



Traverse:

```
for (Node* curr = head; curr != nullptr;
     curr = curr->next)
```

Always check for `nullptr` before `->`. Access by position: $O(n)$; insert/delete at the front: $O(1)$. **Reverse** (three pointers, then `head = prev`):

```
Node *prev = nullptr, *curr = head;
while (curr != nullptr) {
  Node* next = curr->next;  // 1. save
  curr->next = prev;        // 2. flip
  prev = curr;              // 3. advance prev
  curr = next;              //    advance curr
}
head = prev;
```

popFront: if the list is empty, return; else save the node, move the head, *then* delete.

```
if (head == nullptr) return;
Node* toDelete = head;
head = head->next;  delete toDelete;
```

Bug: `delete head; head = head->next;` uses memory after it was freed.

4. Stacks and Queues

Stack: LIFO, `push/pop/top` all $O(1)$ (undo, call stack, DFS). **Queue**: FIFO, `enqueue/dequeue` $O(1)$ (lines, BFS).

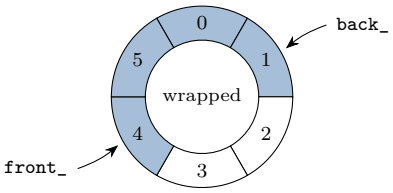
Circular array queue, capacity C (`front_` = first item, `back_` = last item):

- **Empty**: `front_ == back_ == -1`, size 0.
- First enqueue: `front_ = back_ = 0`. Later enqueue: `back_ = (back_+1) % C`.
- Dequeue: `front_ = (front_+1) % C`; if the queue becomes empty, reset both to -1.
- Full when size $= C$ (check before enqueue).

Size (queue not empty): count the items in the index range.
Elements in $[a, b]$ is $b - a + 1$; in (a, b) it is $b - a - 1$.

	Not wrapped	Wrapped
Condition	`back_` $\geq$ `front_`	`back_` $<$ `front_`
Size	`back_` $-$ `front_` $+ 1$	$C -$ `front_` $+$ `back_` $+ 1$

Wrapped = (indices `front_`..$C-1$) + (indices 0..`back_`). Exception: if `front_ == -1`, size is 0.



Shaded = items. Here the items run `front_`..$C-1$ and then 0..`back_`, so `back_ < front_` (generic picture; $C = 6$).

Index arithmetic wraps with `% C`; always advance with `(i+1) % C`, never just `i+1`.

5. Recursion

- Every recursive function needs a **base case** (stops) and a **recursive case** that moves *toward* the base case.
- **Leap of faith**: assume the recursive call works for the smaller input; then just combine its result.
- Each call gets its own **stack frame** (own parameters/locals); frames are popped as calls return. Max stack depth = longest chain of calls *waiting*, not total number of calls.
- **Where the work goes matters:**

```
void show(int n) {
  if (n == 0) return;
  cout << n;   // BEFORE: n, n-1, ..., 1
  show(n - 1);
  cout << n;   // AFTER: 1, 2, ..., n
}
```

- Trace by drawing the call tree: a call prints/returns, then its *siblings* run. Two calls `f(n-1)` and `f(n-2)` in one body run one after the other, never at the same time.
- A missing or wrong base case can cause unbounded recursion and a stack overflow.

6. P2: DFS and BFS on a Maze

Maze: `vector<vector<int>>`, `0` = open, `1` = wall; `Coord{row,col}`; moves are up/down/left/right; stay in bounds, skip walls and visited cells.

	DFS	**BFS**
Structure	stack / recursion	queue
Explores	one path as deep as possible, then backtracks	layer by layer (distance 1, 2, 3, ...)
Visited marked	on *entering* the call	when *enqueued*
Result	first route found (may be long)	**shortest path** (fewest moves)
Path	built while returning	`parent` table, then walk back from goal
Time	$O(n)$ cells	$O(n)$ cells

- **Visited $\neq$ on the route**: visited includes dead ends explored along the way.
- BFS parent table: start's parent is a sentinel `(-1,-1)`; rebuild the path from goal back to start, then reverse it.
- Each cell is visited *at most once*, so both are $O(n)$ for n cells.
- **Design**: `Maze_Solver` is an **abstract** base class with `virtual getPath() = 0`; `DFS_Solver` and `BFS_Solver` override it. A non-virtual `reachable()` that calls `getPath()` still dispatches to the derived version at run time.

7. Big-O Patterns

Code shape	**Time**
Single loop to n; or loops one after another	$O(n)$
Loop inside a loop (both to n)	$O(n^2)$
Inner loop runs 0..i (triangular)	$O(n^2)$
`i *= 2` or `i /= 2` each step	$O(\log n)$
Digits of n (`n /= 10`)	$O(\log n)$ $(= O(d))$
`push_back` with doubling	$O(1)$ amortized
BFS/DFS on n cells	$O(n)$

Drop constants and lower-order terms: $3n^2 + 5n + 2 \in O(n^2)$. Sequential blocks *add*, nested blocks *multiply*.

Quick Checks Before You Submit

- Base case present? Does each call get closer to it?
- Each `new` is released exactly once; never use a pointer after `delete`.
- Check `nullptr` before `->`. Update `head` when the first node changes.
- Queue indices use `% C`; the empty state is -1.
- Polymorphism needs `virtual` *and* a pointer or reference.